EXPERIMENT 5: ELECTION ALGORITHM

1. AIM:

To study and understand election algorithms used in distributed systems for selecting a coordinator (leader) among multiple processes, especially in the presence of failures and dynamic system conditions.

1.1. OBJECTIVES:

1. To understand the concept of election algorithms in distributed systems.
2. To study the role of coordinator (leader) selection among multiple distributed processes.
3. To understand how election algorithms handle process failures in a distributed environment.
4. To analyze the behavior of election algorithms under dynamic system conditions.
5. To study the working of different types of election algorithms such as Bully Algorithm and Ring Election Algorithm.
6. To implement and verify the correctness of election algorithms through practical execution.

1.2. SCOPE OF THE STUDY:

The scope of this study is to understand the fundamental concepts of election algorithms used in distributed systems for selecting a coordinator among multiple processes. It focuses on the mechanisms used to achieve leader selection in the absence of a central authority and examines how these algorithms ensure correct coordination during process failures and system changes.

This study covers the role of election algorithms in handling failures within distributed systems. It emphasizes how the failure of a coordinator process triggers an election and how a new coordinator is selected without disrupting system operation. The scope also includes understanding fault tolerance, recovery mechanisms, and the ability of distributed systems to continue functioning under dynamic conditions.

The scope of this study extends to the application of election algorithms in real-world distributed systems such as cloud computing platforms, distributed databases, and clustered environments. It highlights how leader election contributes to efficient resource management, synchronization, and coordination among distributed components, ensuring reliable system performance.

This study focuses on analyzing different election algorithms used in distributed systems and comparing their working principles, efficiency, and reliability. The scope includes understanding how various algorithms respond to failures, message delays, and changes in system configuration, helping in selecting suitable algorithms for different distributed environments.

2. LITERATURE SURVEY:

Leader election has been a fundamental problem in distributed systems and has been widely studied by researchers. Chang and Roberts (1979) proposed one of the earliest leader election techniques known as the **Ring Election Algorithm**, where processes are logically arranged in a ring and participate equally in the election process. This algorithm ensured fairness but suffered from higher message delays in large systems.

Later, Garcia-Molina (1982) introduced the **Bully Algorithm**, which selects the coordinator based on the highest process identifier. This algorithm improved election speed in smaller systems but increased message complexity as the number of processes grew. The Bully Algorithm became one of the most commonly discussed election techniques due to its simplicity and effectiveness.

Tanenbaum and Van Steen (2017), in their work on distributed systems, provided a detailed explanation of election algorithms and highlighted their importance in fault tolerance, synchronization, and system coordination. They also discussed practical challenges such as network delays, process crashes, and dynamic membership.

Lamport (2001) contributed foundational concepts related to distributed coordination, including logical clocks and ordering of events, which indirectly support leader election mechanisms by ensuring consistency and synchronization across distributed processes.

Recent studies show that election algorithms are widely applied in modern distributed environments such as cloud platforms, distributed databases, and container orchestration systems, where reliable leader selection is essential for maintaining availability and consistency.

Several studies in distributed systems literature describe practical scenarios where election algorithms are applied. For example, Chang and Roberts illustrated a distributed environment in which processes arranged in a logical ring must elect a leader after coordinator failure, ensuring that the process with the highest identifier becomes the new coordinator. Similarly, Garcia-Molina demonstrated through examples how the Bully Algorithm efficiently selects a leader by allowing higher-priority processes to take control when lower-priority coordinators fail.

2.1. THEORY

Distributed Algorithm is an algorithm that runs on a distributed system. Distributed system is a collection of independent computers that do not share their memory. Each processor has its own memory and they communicate via communication networks. Communication in networks is implemented in a process on one machine communicating with a process on another machine. Many algorithms used in the distributed system require a coordinator that performs functions needed by other processes in the system.

Election algorithms are designed to choose a coordinator.
Election Algorithms: Election algorithms choose a process from a group of processors to act as a coordinator. If the coordinator process crashes due to some reasons, then a new coordinator is elected on other processor. Election algorithm basically determines where a new copy of the coordinator should be restarted. Election algorithm assumes that every active process in the system has a unique priority number. The process with highest priority will be chosen as a new coordinator. Hence, when a coordinator fails, this algorithm elects that active process which has highest priority number.Then

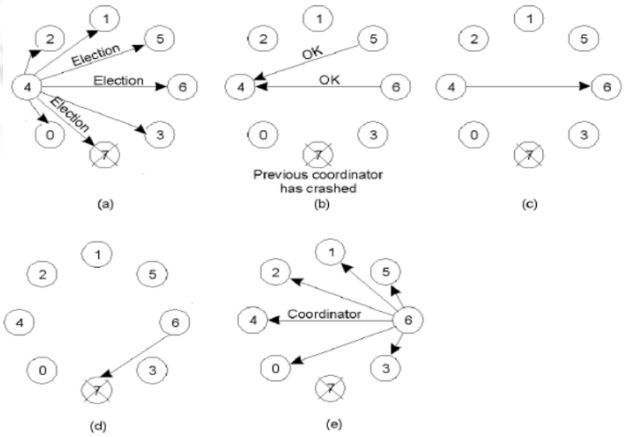
this number is send to every active process in the distributed system. We have two election algorithms for two different configurations of a distributed system.

Election algorithms are triggered when a process detects the failure of the current coordinator, when a process recovers after failure, or when a new process joins the system. Once initiated, the algorithm involves communication between processes using election and acknowledgment messages. The algorithm continues until all active processes agree on a single coordinator. The newly elected coordinator then announces its role to the rest of the system to restore normal operation.

Two widely used election algorithms are the **Bully Algorithm** and the **Ring Election Algorithm**. In the Bully Algorithm, a process that detects coordinator failure sends election messages to all processes with higher identifiers. If no higher-identifier process responds, the initiating process declares itself as the coordinator. In contrast, the Ring Election Algorithm organizes processes in a logical ring, where an election message circulates through all processes, and the process with the highest identifier is selected as the coordinator.

1)The Bully Algorithm - This algorithm applies to system where every process can send a message to every other process in the system. Algorithm **-** Suppose process P sends a message to the coordinator.

1. If the coordinator does not respond to it within a time interval T, then it is assumed that coordinator has failed.
2. Now process P sends an election messages to every process with high priority number.
3. It waits for responses, if no one responds for time interval T then process P elects itself as a coordinator.
4. Then it sends a message to all lower priority number processes that it is elected as their new coordinator.
5. However, if an answer is received within time T from any other process Q,
   - Process P again waits for time interval T' to receive another message from Q that it has been elected as coordinator.
   - If Q doesn't responds within time interval T' then it is assumed to have failed and algorithm is restarted.



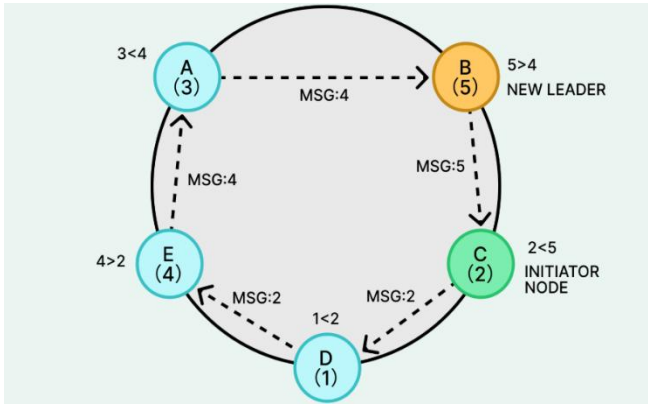
When a process (for example, Process 4) detects that the previous coordinator has failed, it initiates an election by sending ELECTION messages to all processes with higher identifiers. These higher-ID processes respond with OK messages to indicate that they are alive. Any process that receives an

election message but has a higher identifier than the sender takes over the election process and sends election messages to even higher-ID processes.

This process continues until the highest active process does not receive any reply from higher-ID processes. That process then declares itself as the new coordinator and broadcasts a COORDINATOR message to all other processes. This algorithm is called the Bully Algorithm because higher-ID processes always dominate lower-ID processes and take control of the election.

2)The Ring Algorithm - This algorithm applies to systems organized as a ring(logically or physically). In this algorithm we assume that the link between the process are unidirectional and every process can message to the process on its right only. Data structure that this algorithm uses is **active list**, a list that has a priority number of all active processes in the system.

1. If process P1 detects a coordinator failure, it creates new active list which is empty initially. It sends election message to its neighbour on right and adds number 1 to its active list.
2. If process P2 receives message elect from processes on left, it responds in 3 ways:
   - If message received does not contain 1 in active list then P1 adds 2 to its active list and forwards the message.
   - If this is the first election message it has received or sent, P1 creates new active list with numbers 1 and 2. It then sends election message 1 followed by 2.
   - If Process P1 receives its own election message 1 then active list for P1 now contains numbers of all the active processes in the system. Now Process P1 detects highest priority number from list and elects it as the new coordinator.



In the diagram, Process C (ID = 2) detects the failure of the coordinator and starts the election by sending an ELECTION message containing its identifier to the next process in the ring. Each process that receives the message compares the received identifier with its own identifier. If its own identifier is higher, it replaces the identifier in the message before forwarding it further.

This election message circulates through all processes in the ring. When the message returns to the initiator, the identifier with the highest value is selected as the coordinator. In the given diagram, Process B (ID = 5) has the highest identifier, so it is elected as the new leader. After election, a COORDINATOR message is circulated around the ring to inform all processes about the new coordinator.

2.2. Mathematical Treatment:

For Bully Algorithm:

Consider a distributed system with five processes.
Each process has a unique process identifier and a status indicating whether it is active or failed.

Process Number	Process Identifier	Process Status
P1	101	Active (1)
P2	102	Active (1)
P3	103	Failed (0)
P4	104	Active (1)
P5	105	Active (1)

The election is initiated by Process with Identifier = 102.

Solution: Step 1: Define the Set of Processes

Let the set of all processes be:

$$P = \{101,102,103,104,105\}$$

Let the set of active processes be:

$$P_{active} = \{101,102,104,105\}$$

Step 2: Apply Election Condition

According to the Election Algorithm, the coordinator is selected as the active process with the highest identifier, such that:

$$\text{Coordinator} = \max\,(\mathrm{P}_{\text{active}})$$

Substituting the values:

$$\text{Coordinator} = \max\{101,102,104,105\}$$
$$\text{Coordinator} = 105$$

Step 3: Verify Using Bully Algorithm Logic

- Initiator Process = 102
- Higher active processes than 102 are:

$$\{104, 105\}$$

- Process 105 is the highest active process
- No active process exists with an identifier greater than 105

Therefore, Process 105 declares itself as the coordinator.

Final Result:

Coordinator=105

For Ring Election Algorithm:

Consider a ring of four processes with the following identifiers:

$$\mathrm{P} = \{21,45,12,30\}$$

Election is initiated by Process with Identifier = 21.

Step 1: Election Message Circulation
The election message circulates around the ring and collects identifiers:

$$\text{ElectionMessage} = \{21,45,12,30\}$$

Step 2: Apply Coordinator Selection Rule

$$\text{Coordinator} = \max(P)$$
$$\text{Coordinator} = \max\{21,45,12,30\}$$
$$\text{Coordinator} = 45$$

Final Result:

$$\boxed{Coordinator = 45}$$

2.3. ALGORITHM

Algorithm for Bully Algorithm

START

1. A process detects coordinator failure
2. It sends ELECTION message to all higher-ID processes
3. If no reply is received
   - Declare itself as coordinator
4. Else
   - Higher-ID process continues election
5. Coordinator sends COORDINATOR message to all

STOP

Algorithm for Ring Algorithm:

START

1. A process detects coordinator failure
2. Sends ELECTION message to next process in ring
3. Each process adds its ID
4. After full rotation, highest ID is selected
5. Coordinator message circulates

STOP

2.4.PROGRAMMING LANGUAGE

C++ Programming Language
The experiment is implemented using the C++ programming language on a UNIX/Linux operating system. C++ provides strong support for object-oriented and procedural programming, making it suitable for implementing distributed system algorithms. The program uses standard input–output libraries and control structures to simulate the behavior of election algorithms such as coordinator selection among multiple processes. Although the implementation does not rely on low-level kernel system calls, it effectively demonstrates process coordination, message passing logic, and leader election mechanisms in a distributed system environment**.**

2.4.1. Libraries Used

1) <iostream>
   This library provides standard input and output stream objects such as cin and cout.
   It is used to display messages on the screen and to read user input from the keyboard.

2) <algorithm>
   This library provides commonly used algorithms such as finding the maximum value in a collection.
   It can be used to determine the highest process identifier during coordinator selection.

3) <vector>
   This library provides a dynamic array container to store process identifiers and their status.
   It allows flexible handling of process information when the number of processes is not fixed.

3.Implementation

3.1.PROGRAM For Bully Algorithm:

```
#include <iostream>
using namespace std;

int main() {
    int numberOfProcesses;
    cout << "Enter the number of processes: ";
    cin >> numberOfProcesses;

    int processId[numberOfProcesses];
    int processStatus[numberOfProcesses];

    cout << endl;
    for (int index = 0; index < numberOfProcesses; index++) {
        cout << "Enter the ID of process " << index + 1 << ": ";
        cin >> processId[index];
    }

    cout << endl;
    for (int index = 0; index < numberOfProcesses; index++) {
        cout << "Enter status of process " << processId[index]
            << " (1 for active, 0 for failed): ";
```

```
        cin >> processStatus[index];
    }

    cout << endl;
    int initiatorProcess;
    cout << "Enter the ID of the process which initiates the election: ";
    cin >> initiatorProcess;

    cout << endl;
    int coordinator = -1;

    for (int index = 0; index < numberOfProcesses; index++) {
        if (processId[index] >= initiatorProcess && processStatus[index] == 1) {
            coordinator = processId[index];
        }
    }

    if (coordinator != -1) {
        cout << "The coordinator elected using Bully Algorithm is Process "
             << coordinator << endl;
    } else {
        cout << "No active process available for election." << endl;
    }

    return 0;
}
```

OUTPUT:

```
Enter the number of processes: 5

Enter the ID of process 1: 101
Enter the ID of process 2: 102
Enter the ID of process 3: 103
Enter the ID of process 4: 104
Enter the ID of process 5: 105

Enter status of process 101 (1 for active, 0 for failed): 1
Enter status of process 102 (1 for active, 0 for failed): 1
Enter status of process 103 (1 for active, 0 for failed): 0
Enter status of process 104 (1 for active, 0 for failed): 1
Enter status of process 105 (1 for active, 0 for failed): 1

Enter the ID of the process which initiates the election: 102

The coordinator elected using Bully Algorithm is Process 105
```

Program for Ring Algorithm:

```
#include <iostream>
using namespace std;

int main() {
    int numberOfProcesses;
    cout << "Enter the number of processes: ";
    cin >> numberOfProcesses;

    int processId[numberOfProcesses];

    cout << endl;
    for (int index = 0; index < numberOfProcesses; index++) {
        cout << "Enter the ID of process " << index + 1 << ": ";
        cin >> processId[index];
    }

    cout << endl;
    int initiatorProcess;
    cout << "Enter the ID of the process which initiates the election: ";
    cin >> initiatorProcess;

    cout << endl;
    int coordinator = processId[0];

    for (int index = 1; index < numberOfProcesses; index++) {
        if (processId[index] > coordinator) {
            coordinator = processId[index];
        }
    }

    cout << "The coordinator elected using Ring Election Algorithm is Process "
         << coordinator << endl;

    return 0;
}
```

OUTPUT:

```
Enter the number of processes: 4

Enter the ID of process 1: 21
Enter the ID of process 2: 45
Enter the ID of process 3: 12
Enter the ID of process 4: 30

Enter the ID of the process which initiates the election: 21

The coordinator elected using Ring Election Algorithm is Process 45


=== Code Execution Successful ===
```

CONCLUSION

The Election Algorithm was successfully studied and implemented. The experiment demonstrated how a coordinator is elected in a distributed system when the existing coordinator fails. The Bully Algorithm efficiently selects the highest-ID process, while the Ring Algorithm ensures fair participation of all processes. This experiment highlights the importance of election algorithms in maintaining coordination, fault tolerance, and reliability in distributed systems.

Sign and Remarks:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature